UNIT-3

__/__/____

Process Synchronization

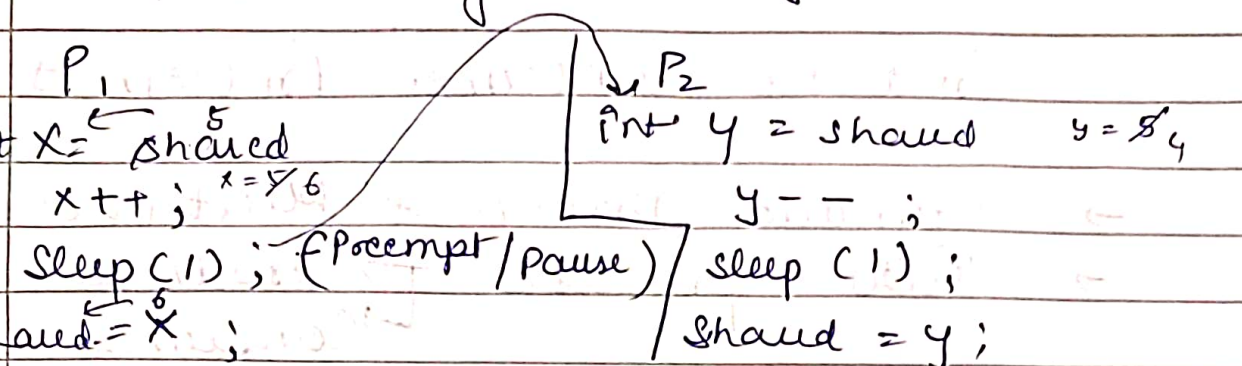
Two modes of execution → Serial
→ Parallel
(effect of one execution affects another process.)

Process
- Cooperative Process
  - * Execution of one process affects the execution of another process
  - bcoz they share → variable
    - → Memory/buffer
    - → Code
    - → Resources → CPU, Printers, Scanner
  - ex. * IRCTC website ticket booking transactions.
- Independent (Nothing in common)
  - * execution of one process does not affect execution of another process.
  - ex. transaction of two HDFC & ICICI Bank

* Cooperative processes may create a problem if not synchronized properly then they create an problem.

int shared = 5 6

two process running on same system . (uniprocessor)



Sleep (1) = preempt / pause for 1 sec

if P1 executes first & then P2 executes
new shared value = 5 6 4

Process P1 & P2 are cooperative but not synchronized

* this condition is called race condition.
* If process are not synchronized then they give inconsistent or invalid result.

__/__/____

Producer Consumer Problem :

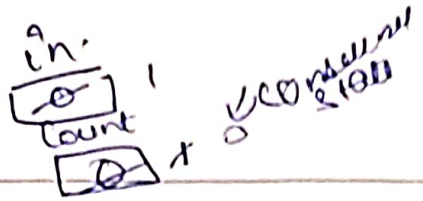
→ Standard problem of Multiprocessor
→ Two processes → producer
→ consumer

Assumption → Both arrive at same time i.e. parallel & cooperative processes

Producer produces item and place them in buffer and consumer consumes that item.

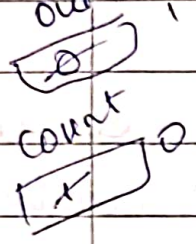
Producer :

```
int count = 0;
void producer (void)
{
int itemp
while (true)
{
  produce_item (itemp);
while (count == Buffer_size);
buffer [in] = itemp
  in = (in+1) % buffer_size;
count = count + 1;
}
```



Consumer:

```
Void Consumer (Void)
  {
   int itemc;
  while (true)
   {
Buffer_item ← { while (count == 0);
                itemc = buffer (out);
                Out = (Out + 1) mod buffer-size.
                 count = count - 1
                 process (itemc);
              }
```



We are inserting item from 0 & taking out them from 0 position

Best case Case 1: Producer & Consumer work one after another.

Case 2: While producer is producing but in mid consumer consumes it

Critical Section :-

→ It is a part of program where shared resources are accessed by various processes.

* It is a portion of program where shared resources, shared variable & shared data are placed.

* Each process has a critical section where it changes common variable, update table.

* No two process can be in critical section at a same time.

* Non common (→ independent) data can be placed in Non critical section.

do {

Entry Section → Each process must request process to enter in its CS.

semaphor
lock
Monitors.

Critical Section

Exit Section

Remainder Section → Remaing code ie other than shared variable/data.

} while(true);

4 Rules/Condition to achieve process Synchronization.

→ Must fulfil these 4 Condition

1. Mutual Exclusion } primary
2. Progress
3. Bounded Wait
4. No assumption related to H/W speed.

Solution TO CS

Software solution
- Peterson's
- Dekker's

H/w solution
- Test & Set
- Swap

Operating System solution
- Counting semaphore
- Binary "

Computer & programming support type
* Monitors

let turn = 1 initially

ex.

P_0	P_1
while (1) {	while (1) {
while (turn != 0);	while (turn != 1);
CS	CS
turn = 1;	turn = 0;
remainder section }	remainder section }

So this code ① ME ✓
② Progress ✗

Strict alternation.

* CS solve using lock:

```
do{ acquire lock        Entry
      CS                section
    release lock
}
```

```
1. while (lock == 1);   <- True goes to infinite loop
2. lock = 1
3. CS
4. lock = 0;            Exit code
```

* Execute in user mode
* Multiprocess solution
* No mutual exclusion guarantee.

lock = 0 initially → vacant

1 → full..

Case 1.

P_1	P_2
1. 2. 3. 4.	× lock = 0 + 0 1
lock → 1 2. 3 4	
exit c.s.	

No guarantee of ME

Case 2.

P_1	P_2	L = 0
1 * preempt before line 2.	1 2 3 c.s.	L = 0 + 1
when P_1 resumes then make lock = 1		P_2 ✓ c.s.

1 * 2 3

* which makes P_1 & P_2 enter in c.s

Test & Set

lock = 0 — vacant

= 1 → full

Case 1 :

→ use H/w.

CS Solut' using test & set instruct'n

Combine 1 & 2 inst'.

false → funct'

while(lock==1) lock=1 → while (test_and_set (& lock));

& use H/w

CS :

lock = false ;

→ call by reference.

initial lock = false

```
boolean test_&_set (boolean *target)
{
    boolean r = *target;
    *target = TRUE;
    return r;
}
```

false

pointer variable

ME ✓

Progress ✓

lock	target	r
false / True / false	1000 (Addr)	False

ME ✓

Progress ✓

for P2

P1 P2

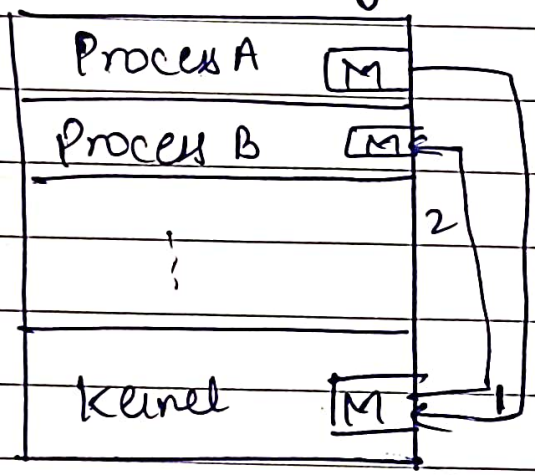
Initially lock = false

__/__/__

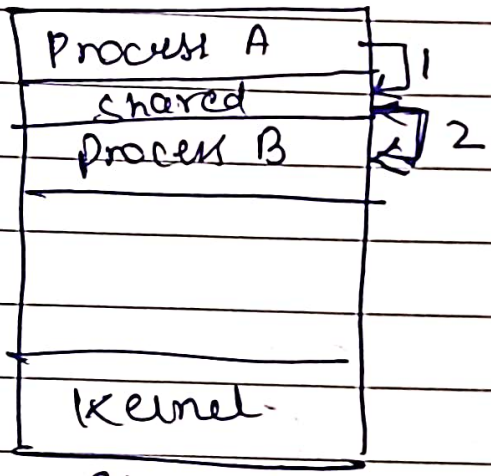
Inter process communication →

It is a mechanism that allow processes to communicate with each other and synchronize their actions. There are two Modes of IPCs →

1. Shared Memory
2. Msg passing



Msg passing



Shared Memory

IPC Msg passing →

→ Mechanism for process to communicate & to synchronize their actions

→ Msg system - process communicate with each other without resorting to shared variable

Date: __/__/____

UNIT-3

Turn Variable (Strict alteration)

→ Run in user mode
→ 2 process solution

int turn = 0	initially.
P_0	P_1
entry code 1. while (turn != 0);	while (turn != 1);
2. CS	CS
exit code 3. turn = 1;	Turn = 0;

ex. int turn = 0
execute P_0 first if turn = 0 get false in 1. Step. then P_0 enters in CS.

- ME is satisfied here.
- Progress X
  as if we take P_1 = turn = 0, initially then P_1 will not get execute. Only if first P_0 is executed after that P_1 will execute.

Date: __/__/____

* Bounded wait ~ is satisfied
* Not dependent on H/w & S/w.
* as if Po executes then P1 with execute after that. due to Changes in turn variable or vice versa. this is also known as Strict alteration.

Date: __/__/____

Semaphore → It is a method or tool to prevent race condition

Semaphore is an integer variable which is used in mutual exclusive manner by various concurrent cooperative processes in order to achieve synchronization.

Semaphore

Counting	Binary
integer value ($-\infty$ to $+\infty$)	integer value will be (0 or 1)

Operations → P() , Down , Wait (Entry code)
V() , Up , Signal , Post , Release (Exit code)

entry code

```
Down (semaphore S)
{
S.value = S.value - 1
if (S.value < 0)
{
put process PCB in
suspend list sleep();
}
else
return;
}
```

exit code

```
Up (semaphore S)
{
S.value = S.value + 1
if (S.value ≤ 0)
{
select process from
suspended list
wake up();
}
}
```

Ex. $S = 3$

P_1 P_2 P_3 P_4 P_5 (Blocked State)

$3, 2, 1, 0, -1, -2$ | P_4 P_5

CS P_1 P_2 P_3

ex. S = -4 means, 4 process are in blocked state.

Date: __/__/____

$S = 0$ means further proces will not enter in CS.

* After 0 NO process will enter CS they will go to blocked state.
* Every process has to exit CS using exit code.

ex. if P_1 wants to exit CS.

P_1 P_2 P_3 . P_4 P_5

3 2 1 0 −1 (−2) → (+1)

= −1

* wakeup in done using FIFO

ex. $S = 10$ how many proces succesfully enter CS.

Ans = 10

ex. $S = 10$

6 P + 4 V operation what is value of Semaphore.

Solut^ 10 − 6 + 4 ie. S − P + V

Que S = 17 5 P, 3 V, 1 P

solut^ S − P + V

17 − 5 − 1 + 3 = 14

#learnthesmarterway

Date: __/__/____

Binary Semaphore:

```
Down (Semaphore S)
{
if (S.Value == 1)
  {
  S.Value = 0;
  }
else
{
block this process and
place in suspend list
Sleep();
}
}
```

```
Up (Semaphore S)
{
  if (suspend list is
      empty)
  S.
  S.value = 1;
  }
else
{
select a process from
suspend list
wakeup();
}
}
```

for Down: S = +0 Successful Operation

S = 0 Unsuccessful Operation enter in suspend list.

let S=0

Date: __/__/____

P_1	P_2
Down (S)	Down(S)
CS	CS
Up(S)	Up(S)

Solut: P_1 is executing first

P_1 goes in blocked state. result in deadlock state.

ex. case 2: S=1

let P_2 is executing first.

Now, S=0 & P_2 enter CS.

Now P_1 comes.

So P_1 enter in block state.

Date: __/__/____

Solut^n of Producer Consumer using Binary & counting Semaphore.

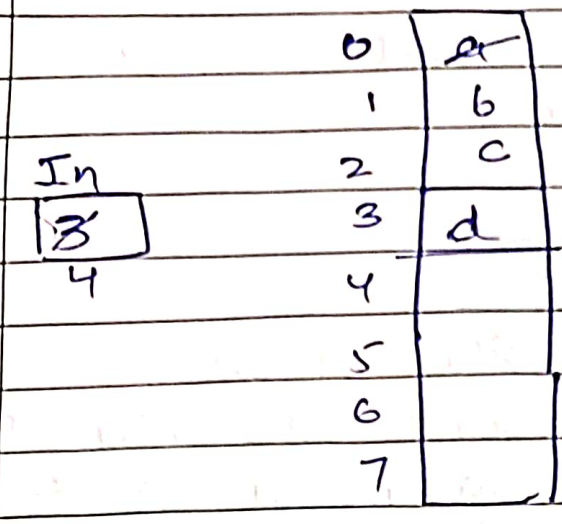
& - Counting → Full = 0 (No. of full slots)
Empty = N (No. of empty slots)

Binary semaphor S = 1

Producer	Consumer
produce_item (item p);	down (full);
1. down (Empty) ;	down (S);
2. down (S);	
cs: Buff [In] = item p;	item C = Buff[out];
In = (In+1) mod n;	out = (Out+1) mod n;
3. up(S);	up(S);
4. up (full);	up(Empty);
	=

N = 8

Date: __/__/____



Empty = ~~5~~ 4

full = 3

S = ~~1~~ 0 → enter a item

In = (3+1) mod 8 = 4 mod 8

= 4

at 3 step S = ~~0~~ 1

at 4 step full = ~~3~~ 4

- Work well sequentially like P_4 executes first & Consumer executes.

Case 2: ~~Both~~ if preemption take place

Empty = 5

full = 3.

Producer preempt before step 2.

It creates inconsistency; so using semaphore we execute consumer first & then execute producer

Date: __/__/____

1. Two process solution with flag variable

	0	1
flag	F	F

initially Flag = false
If process wants to enter CS than its flag must be true.

P_0	P_1
while (1)	while (1)
{	{
1. flag [0] = T;	flag [1] = T;
check flag P_1 2. while (flag [1]);	while (flag [0]);
3 CS	CS
3. flag [0] = F	flag [1] = F
}	}

	0	1
flag	F T	F

Date: __/__/____

* It satisfy ME
* progress ✗ if preempt P0 at after step ①. which block both P1 & P0.

	0	1
flag	F	T
	T	T

leads to deadlock at same timing sequence.

3. Bounded Wait

Peterson Solution":

⇒ use Both turnVariable and flag away

P0	P1
while (1)	while (1)
{	{
flag[0] = T	flag[1] = T
turn = 1	turn = = 0
while (turn == 1 && flag[1]=T);	while (turn == 0 && flag[0] == T); ← P1
CS ← P0	CS
flag[0] = F	flag[1] = F
}	}

if false exit while loop

Date: __/__/____

ex. turn = 1

flag [F | F] } initially
0 1

if Po executes first

Turn = 1

flag [F T | F]

after Po exit

Turn = 0

flag [F | T]
0 1

1. Ensures Mutual Exclusion

Turn = 1

flag [F | F]

- Po execute multiple times Check condit" satisfy Po execute multiple times.
- if both Po & P1 execute concurrently

Turn = 1 or 0

flag [F T | F T]

P1 enter first turn is equal to 1 at step 3

Date: __/__/____

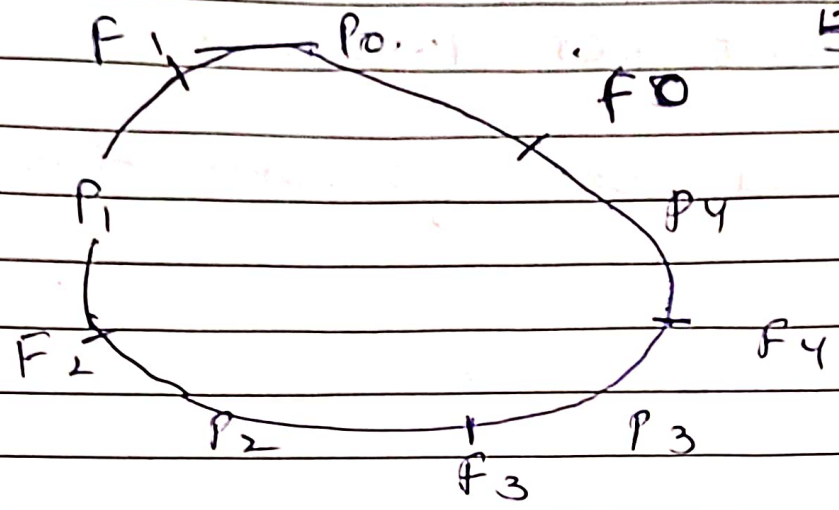
2. Satisfy progress
3. Bounded wait. ✓
   Wait for 1 round.

It is good for only two processes.

Date: __/__/____

Dining Philosophers

5 philosophers
5 Chopstic




```
while (True)
{
Thinking ();
wait (take_fork(i));}        ← left fork
wait (take_fork((i+1) % N));  Right fork
Eat();
signal (Put fork(i));
signal (Put fork((i+1)% N));
}
```

N → No. of forks

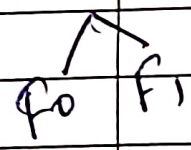
Date: __/__/____

They have three states eating, thinking hungry;

* First take left fork then take right fork.
→ Then start eating
→ After eating put left fork & then put right fork.

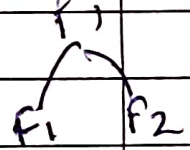
Case 1: P0

P0 pick first F0 fork then pick (i+1) % 5)
i.e. (0+1) % 5
= F1



$P_0 \rightarrow F_0\ F_1$
P0 put back both forks.

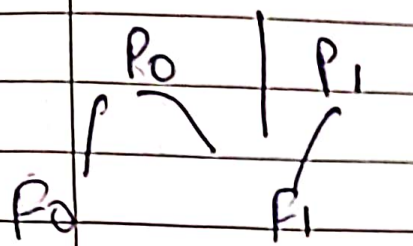
then P1 comes
and Put f1 and f2
and put back both



this code work well if executes sequentially.

Date: __/__/____

Case 2: P0 first Pick f0 and then P1 comes and Pick f1 then P0 is waiting for f1



Solut^: We will use semaphore. S[i] ie. 5 semaphore

S0 S1 S2 S3 S4

and all all inihized with 1.

S0	S1	S2	S3	S4
1	1	1	1	1

P0 →	S0 S1
P1 →	S1 S2
P2 →	S2 S3
P3 −	S3 S4
P4 −	S4 S0

Date: __/__/____

P0 P1 P2
↓ ↓
Block

easily enter CS

S0	S1	S2	S3	S4
1	1	1	1	1
+	+	+	+	+
0	0	0	0	
P0		P2		

* Allow 2 procen in CS but they must be independent. (P0, P2)

Case 3.

	S0	S1	S2	S3	S4
P0	+	1	1	1	1

No one is eating
This case has deadlock situat^n
if P0 comes first take S0 left fork and preempt before picking right fork. and P1 comes.
P1 again take left fork and preempt before picking right fork and P2 comes. and demanding same this will lead to deadlock.

Solut^n

Solut' Code of $(N-1)^{th}$ (Same.)

Date: __/__/____

Code for N^{th}

wait (take fork (S_{i+1} mod N)
wait (take fork (S_i)
eat();
signal (put fork ($(i+1)$ % N)
signal (put fork (i);)

Date: __/__/____

Reader Writer Problem :

issue occur on same data

R	W	Problem
R	R	No issue
W	R	Problem
W	W	Problem

* Semphore mutex = 1
Semaphore db = 1
No of reader using database

RC → Read Count

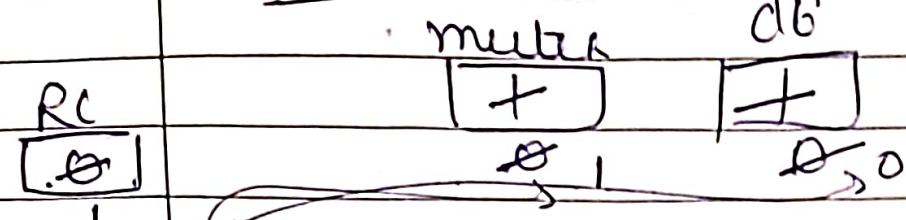
```
int rc=0
Semaphore mutex=1;
Semaphore db=1;
void reader()
{
 while (true)
 {
 down (mutex);
 rc=(rc+1) ;
if (rc=1) then down db;
   up mutex;
 DB (CS)
down mutex
rc = rc-1;
if(rc==0) then up(db)
up mutex)
Process data
 }
 }
```

#learnthesmarterway

R - W.

Date: __/__/____

Case 1: R1 come first



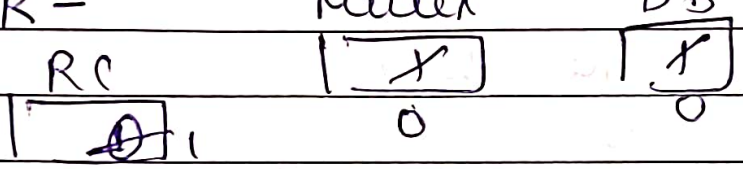
rc = rc+1 i.e rc = 1

if (rc == 1) down(db)
i.e db = 0

Up (mutex)
Now R1 enter DB or C.S

at same time writer comes.
down(db) i.e. db is already 0.
Now writer will Not enter C.S.

Case II W-R-



W enter DB or C.S.

Now, R comes

Mutex = 0 RC = rc+1 = 1

(RC = 1) and then down(db)
But DB already 0.

So No reader will come.

Date: __/__/____

```
Void writer ()
{
while (true)
{
down (db);
 B
up (db);
}
}
```

Case 3. W-W

Mutex	dB	RC
1	1	0
	0	

W1 enter.
And W2 comes. db is already 0 which blocks writer 2.

Case 4:

Mutex	dB	RC
1	1	0
0	0	2

R1 enter C.S.
Now R2 comes. Mutex is 1,
up mutex.
Now R2 enter C.S. / DB.

#learnthesmarterway

Date: __/__/____

Bakery Algorithm:

- It is a CS solut^n for N processes.
- Preserve FCFS property.
- Each process is assigned a NO. in lexicographical order.
- Before entering C.S., process receives a ticket and process with smallest ticket enters C.S.
- If two processes receive same ticket NO, process with lower proces ID is given priority.

if $i < j$

Pi is served first

else

Pj is served first.

Algo Psuedocode:

Repeat

(intend to enter C.S) choosing[i] = true;

Number[i] = max[number[0], number[1] ... number(n-1)] + 1;

(* First 3 line all used to modify its ticket value & at same time other processes will not check them)

Date: __/__/____

```
choosing[i] = false    (assign new ticket)
for j := 0 to (n-1)
  do begin
  while choosing[j] do no op ;
  while number[j] != 0
and (number[j], j) < (number[i], i) do no op;
    end;
  CS
number (i) = 0;
    remainder Section .
until false ;
```

Adv:

1. fairest
2. Easy to implement
3. No deadlock
4. No. Starvation

Disadv:

1. Not Scalable
2. High time Complexity
3. Busy Waiting
4. Memory overhead.

Date: __/__/____

Monitors :

- Provide high level abstraction for data access & synchronization
- Implemented using programming constructs.
- Provide Mutual Exclusion, condition variable and data Encapsulation in single construct.

- Implemented via multithread systems
- At a given time, only one process can be a part of monitor, Other process must wait until the process, which is already in monitor leaves it.

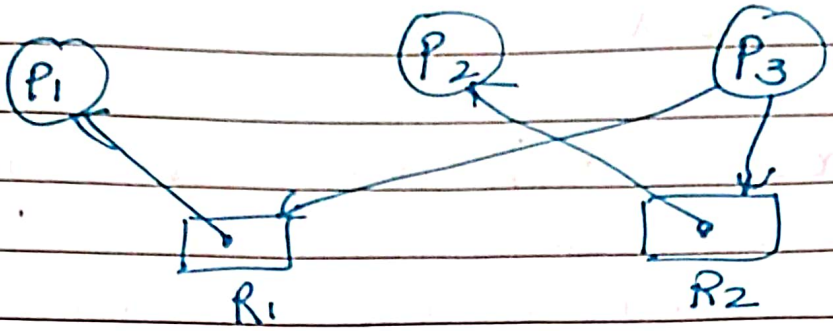
Adv: 1. Make parallel programing easier
2. Less prone to error

Disadv: As monitor are part of programing lang. Compiler must generate code for them. which is a overhead.

Date: __/__/____

Deadlock

Ques.

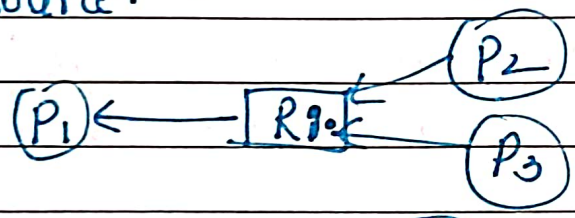


Check for deadlock.

Wait for Graph: (WAG) → Contains only process Nodes

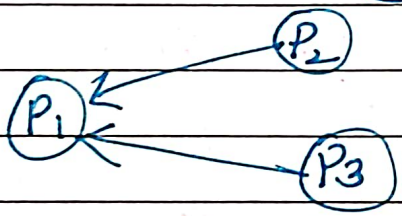
Edge (P_i, P_j) represents that P_i is waiting for process P_j to release a resource.

Ex



RRAG

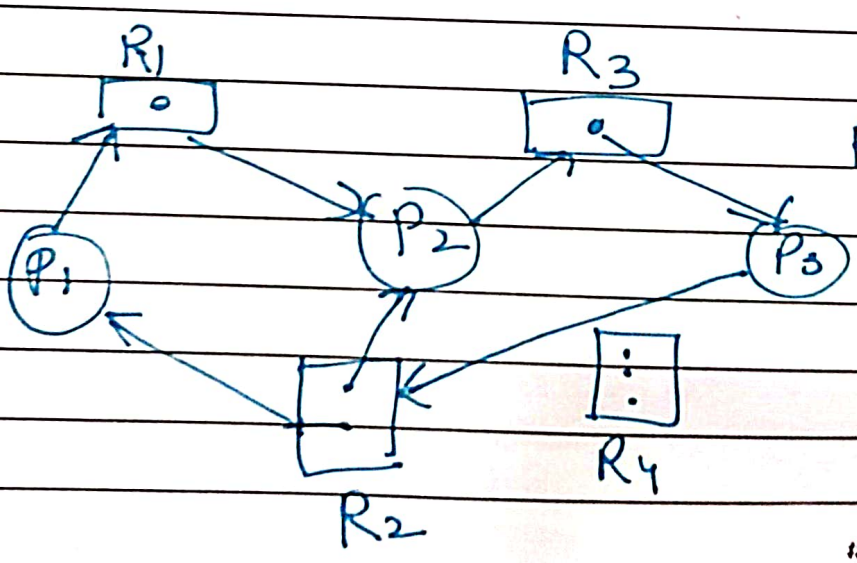
WFG



Date: __/__/____

* If Graph contain No cycle - No deadlock
* if Graph has cycle → Deadlock may exist
* if Resource type has one instance Unit then cycle implies deadlock is occured. Each process in cycle is deadlocked.
* Cycle in a graph is necessary & sufficient condition for deadlock existance
* if resource type has several instances, then cycle in graph does not implies that deadlock has occured
* Cycle is ~~NOT~~ Graph is necessary but not a sufficient condition for deadlock existance.

Ex.

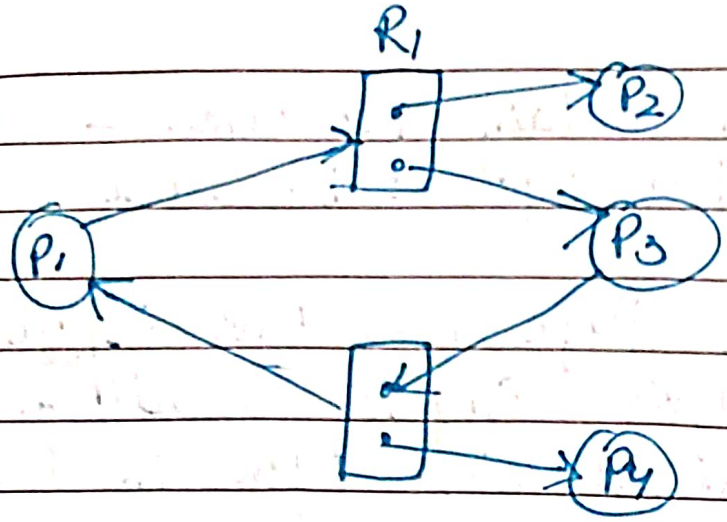


RRAG has deadlock

#learnthesmarterway

Date: __/__/____

Ex:

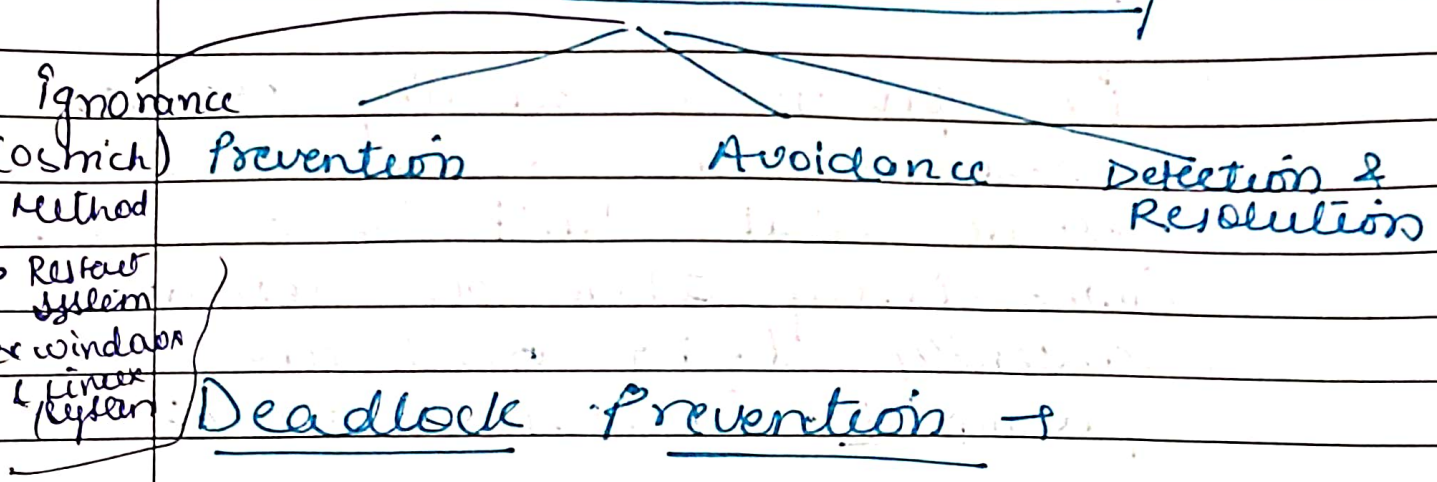


Cycle but No deadlock

→ here P2 and P4 are not requesting for any resource
ie there is a hold but Not wait
After completing their execution they release the hold resources.

Date: __/__/____

Deadlock Handling



Deadlock Prevention →

* Ensure that atleast one of these condition cannot hold; we can prevent deadlock.

1. Mutual Exclusion → some Non resources are Non shareable ex printer. Some are shareable ie read only files.

2. Hold & wait → should Never occur in system.
Whenever a process require resource, it does not hold any other resource.

Date: __/__/____

3. No Preemption :-

if process is holding some resource and request another resource that can't be allocated immediately then all resources currently being held are preempted. (Release resources)

- Preempt resources are added to list of resources for which process is waiting.
- Process restart only if it has old resources as well as new resources that it is requesting.

* can use time quantum.

Date: __/__/____

4 Circular Wait →

Resources can be Numbered.
ex. tape drive – 1
disk. ↑ = 2
printer = 3

A process has access to disk drive & printer. It should access disk drive first & then printer.

* Once a process has some resource allocated to it, it can allocate new resource only if
No. of Allocated resources < No. of assigned to required resource.

Request in increasing order

Date: __/__/____

Deadlock Avoidance :

* Require additional information about how resources are requested.

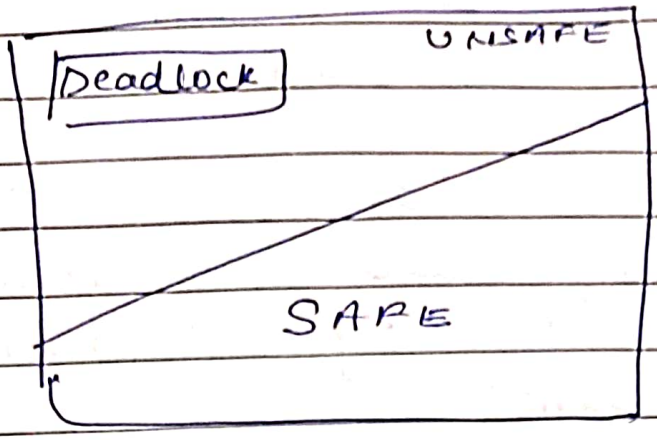
ie. Give prior knowledge of Max. no. of resource class & their resource units. needed in system

→ With this prior information it is possible to construct an algo that ensure that system will Not enter in deadlock.

⇒ This algo examine resource - allocation state to ensure that ~~deadlock~~ circular wait condition can never exist.

Date: __/__/____

Safe State.



A state is safe, if system can allocate resources to each process in some order and avoid deadlock.

- Safe state is not deadlock state.
  Deadlock state is unsafe state.
  Not all unsafe state are deadlock state.
  ie. unsafe state may lead to deadlock state.

Date: __/__/____

1. For a single instance of resource type – Resource allocation Graph is used.

→ New edge called Claim edge is used and represented by $P_i \rightarrow R_j$. (Represented by dashed line)

→ Indicate that P_i may request resource R_j at some time in future.

When P_i request R_j, claim edge $P_i \rightarrow R_j$ is converted to request edge.

When R_j is released by P_i allocation edge $R_j \rightarrow P_i$ is again converted to claim edge $P_i \rightarrow R_j$

Date: __/__/____

Banker's Algo: Applicable to resource allocation system with multiple instances of each resource type.

- Declare Max. No. of resources in advance. (May not exceed the total No. of resource in system)
- When a process request a resource, it may have to wait.
- When a process gets all resources, it must return them in finite amount of time.

Data Structure for Banker's Algo :-

n → No. of processes
m → No. of resource type.

- Available : Vector of length m
  if available [j] = k, it means there are k instances of resource type R_j.

Date: __/__/____

- **Max:-** Matrix of $n \times m$

  if $max[i, j] = k$, process P_i may request atmost k instances of resource type R_j

Allocation → Matrix of $n \times m$

allocation$[i, j] = k$

P_i is currently allocated k instances of R_j.

Need = Matrix of $n \times m$

need$[i, j] = k$, process P_i may need k instances of Resource type R_j.

$$Need[i, j] = max[i, j] - Allocation[i, j]$$

- Banker Algo is used to find Need Matrix.

Date: __/__/____

Safety Algo: Extention of Banker's Algo

→ Used to identify whether system is in safe state or not.

* Used to determine safe sequence.

if need ≤ available
execute process & find new available.

New available = available + allocation
go forward
else
do not execute.

Banker's Algo: A = 10, B = 5, C = 7

Process	Allocation	Max Need	Current Available	(MAX Need − Allocation) Remaining Need
P_0	A B C	A B C	③ A B C	② A B C
P_0	0 1 0	7 5 3	3 3 2	~~7~~ 4 3 ✓
P_2	2 0 0	3 2 2	+2 0 0 → 5 3 2	1 2 2 ✓
P_3	3 0 2	9 0 2	+2 1 1 → 7 4 3	6 0 0
P_4	2 1 1	4 2 2	7 4 ~~3~~ 5	2 1 1 ✓
P_5	0 0 2	5 3 3	7 5 5	5 3 1 ✓
Total ① allocated	7 2 5		+3 0 2 → 10 5 7	

P_1

~~for P_4~~ 3 3 2 − 2 1 1 → 1 2 1

for P_2, P_4, P_5, P_1, P_3

safe sequence.

#Idarnthesmarterway

Date: __/__/__

	Allocation	Max Need	Available	Remaining Need
	E F G	E F G	E F G	E F G
P_0	1 0 1	4 3 1	3 3 0	3 3 0
P_1	1 1 2	2 1 4	4 3 1	1 0 2
P_2	1 0 3	1 3 3	5 7 4	0 3 0
P_3	2 0 0	5 4 1	6 4 6	3 4 1
			8 4 6	

P_0, P_2, P_1, P_3 Safe state

Qu	Process	Allocat^n A B C D	Max. A B C D	Available A B C D
	P_0	0 0 1 2	0 0 1 2	1 5 2 0
	P_1	1 0 0 0	1 7 5 0	
	P_2	1 3 5 4	2 3 5 6	
	P_3	0 6 3 2	0 6 5 2	
	P_4	0 0 1 4	0 6 3 6	

1. Need Matrix

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 7 & 5 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 6 & 4 & 2 \end{bmatrix}$$

P_0 arrives

New Avail = Av. + Allocation
= 1520 + 0012
= 1532

P_1 arrives – Not executed
Need > available.

$P_0, P_2, P_3 P_4 P_1$

Date: __/__/____

P_2 arrives Need ≤ Available

New Ava. = 1532 + 1354

= 2886

Executed

P_3 arrives – Need ≤ Available

0632 ≤ 2886

New avai = 2886 + 0632

= <2 14 11 8>

P_4 arrives – Need ≤ Available

0642 ≤ 2 14 11 8

New Available = <2,14,11,8> + <0 0 1 4>

= <2 14 12 12>

Executed

P_1 arrives.

Need ≤ Available

<0 7 5 0> ≤ <2 14 12 12>

New Available = Available + allocat^n

= <2 14 12 12> + <0 0 0 0>

= <3 14 12 12>

So, Safe sequence is – P_0, P_2, P_3, P_4, P_1